

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №3

Проектирование вычислительных систем

Вариант: 3

Выполнили:

студенты 3 курса

Батманов Даниил Евгеньевич 

Разинкин Александр Владимирович 

Группа: Р3407

Принял:

Пинкевич Василий Юрьевич

Отчёт принят «__»____2025 г.

Оценка: _____

г. Санкт-Петербург, 2025

Цели работы

1. Получить базовые знания об устройстве и режимах работы таймеров в микроконтроллерах.
2. Получить навыки использования таймеров и прерываний от таймеров.
3. Получить навыки использования аппаратных каналов ввода-вывода таймеров.

Задание

Разработать программу, которая использует таймеры для управления яркостью светодиодов и излучателем звука (по прерыванию или с использованием аппаратных каналов). Блокирующее ожидание (функция HAL_Delay()) в программе использоваться не должно.

Стенд должен поддерживать связь с компьютером по UART и выполнять указанные действия в качестве реакции на нажатие кнопок на клавиатуре компьютера. В данной лабораторной работе каждая нажатая кнопка (символ, отправленный с компьютера на стенд) обрабатываются отдельно, ожидание ввода полной строки не требуется.

Для работы с UART на стенде можно использован один из двух вариантов драйвера (по прерыванию и по опросу) на выбор исполнителя. Поддержка двух вариантов не требуется.

Частота синхросигнала процессорного ядра и сигнала ШИМ для управления яркостью светодиодов (если используется) должны соответствовать указанным в варианте задания.

Порядок выполнения работы

1. Изучить:

- разделы учебного пособия: о 1.5. Программируемые таймеры;
 - электрическую принципиальную схему стенда в части излучателя звука;
 - разделы справочного руководства RM0090:
 - o Basic timers (TIM6 and TIM7);
 - o General-purpose timers (TIM2 to TIM5);
 - o General-purpose timers (TIM9 to TIM14);
 - o Advanced-control timers (TIM1 and TIM8);
 - состав стандартных драйверов таймеров из библиотеки HAL и содержимое создаваемых генератором файлов tim.c/h (появляются в проекте после настройки таймеров в графическом конфигураторе).
2. Подготовить шаблон проекта для STM32CubeIDE, настроить тактовые частоты.
 3. Настроить входы и выходы микроконтроллера, таймеры и UART.

4. Разработать необходимые драйверы управляемых светодиодов и/или излучателя звука (зависит от варианта). Подготовить формулы для расчета необходимых частот, после выполнения работы включить их в отчет с объяснением, как они реализуются в программе.
5. Разработать прикладную программу согласно варианту задания и протестировать ее.

Ход работы

Вариант 3

Реализовать имитатор гирлянды с плавными включениями/выключениями светодиодов и переходами между цветами. Должно быть предусмотрено четыре разных предустановленных режима анимации и один пользовательский режим, который можно настраивать. В каждом режиме должно быть задействовано минимум два светодиода, при этом не обязательно, чтобы они могли гореть одновременно.

Действия стенда при получении символов от компьютера:

Символ	Действие
«1»	Переход на следующий режим.
«2»	Возврат к предыдущему режиму.
«3»	Ускорение воспроизведения анимации на 10 % от текущей скорости.
«4»	Замедление воспроизведения анимации на 10 % от текущей скорости.
«5»	Вывод в UART параметров текущего сохраненного пользовательского режима.
«Enter»	Вход в меню настройки.
На усмотрение исполнителей	Ввод параметров пользовательского режима: цвет светодиода, яркость, необходимость плавного перехода, длительность, конец последовательности и т. п.

После ввода каждого символа в UART должно выводиться сообщение о том, какой режим активирован, или текущие настройки, вводимые в меню.

Частота процессорного ядра – 90 МГц, частота ШИМ-сигнала для светодиодов – 4000 Гц.

Counter Settings	
Prescaler (PSC - ...	224
Counter Mode	Up
Counter Period (A...	99
Internal Clock Divi...	No Division
auto-reload preload	Disable
Trigger Output (TRGO) P...	

Исходный код

```
/* USER CODE BEGIN Header */
```

```

/**
*****

* @file      : main.c
* @brief     : Main program body
*****

* @attention
*
* Copyright (c) 2025 STMicroelectronics.
* All rights reserved.
*
* This software is licensed under terms that can be found in the LICENSE file
* in the root directory of this software component.
* If no LICENSE file comes with this software, it is provided AS-IS.
*
*****

*/

/* USER CODE END Header */

/* Includes -----*/

#include "main.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"
#include "string.h"

/* Private includes -----*/

/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/

/* USER CODE BEGIN PTD */

```

```
typedef enum {
    BREATH = 0,
    STROBE,
    SIREN,
    EVERY_SIGNAL,
    CUSTOM_MODE
} mode;
```

```
typedef enum {
    GREEN,
    YELLOW,
    RED
} indicator_color;
```

```
typedef struct {
    indicator_color color;
    uint32_t brightness;
    _Bool is_smooth;
    uint32_t duration;
} custom_mode_step;
```

```
typedef enum {
    MAIN,
    CUSTOM
} menu_state;
```

```
/* USER CODE END PTD */
```

```
/* Private define -----*/
```

```
/* USER CODE BEGIN PD */
```

```
#define MIN_BRIGHTNESS 0
```

```
#define MAX_BRIGHTNESS 100
```

```
#define MAX_CUSTOM_MODE_STEP_COUNT 5
```

```
/* USER CODE END PD */
```

```
/* Private macro -----*/
```

```
/* USER CODE BEGIN PM */
```

```
/* USER CODE END PM */
```

```
/* Private variables -----*/
```

```
/* USER CODE BEGIN PV */
```

```
uint32_t speed_percents = 50;
```

```
mode current_mode = BREATH;
```

```
char* mode_name[] = {
```

```
    "breath",
```

```
    "strobe",
```

```
    "siren",
```

```
    "every signal"
```

```
};
```

```
custom_mode_step custom_mode_steps[MAX_CUSTOM_MODE_STEP_COUNT];
```

```
uint32_t current_custom_mode_step_count = 0;
```

```
uint32_t current_custom_mode_step_index = 0;
```

```
uint8_t input_buffer;
```

```
uint8_t output_buffer[256];
```

```
menu_state current_menu_state = MAIN;
```

```
/* USER CODE END PV */
```

```
/* Private function prototypes -----*/
```

```
void SystemClock_Config(void);
```

```
/* USER CODE BEGIN PFP */
```

```
void wait(uint32_t delay);
```

```
void play_breath();
```

```
void play_strobe();
```

```
void play_siren();
```

```
void play_every_signal();
```

```
void play_custom_mode();
```

```
/* USER CODE END PFP */
```

```
/* Private user code -----*/
```

```
/* USER CODE BEGIN 0 */
```

```
/* USER CODE END 0 */
```

```
/**
```

```
 * @brief The application entry point.
```

```
 * @retval int
```

```
 */
```

```
int main(void)
```

```
{
```

```
/* USER CODE BEGIN 1 */
```

```
/* USER CODE END 1 */
```

```
/* MCU Configuration-----*/
```

```

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM4_Init();
MX_USART6_UART_Init();
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);

HAL_UART_Receive_IT(&huart6, &input_buffer, 1);
while (1)
{

```



```

    if (current_mode == BREATH) {
        play_breath();
    } else if (current_mode == STROBE) {
        play_strobe();
    } else if (current_mode == SIREN) {
        play_siren();
    } else if (current_mode == EVERY_SIGNAL) {
        play_every_signal();
    } else {
        play_custom_mode();
    }

/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
     */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

```

```

/** Initializes the RCC Oscillators according to the specified parameters
 * in the RCC_OscInitTypeDef structure.
 */

```

```

RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState = RCC_HSE_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLM = 15;
RCC_OscInitStruct.PLL.PLLN = 108;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
RCC_OscInitStruct.PLL.PLLQ = 4;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

```

```

/** Activate the Over-Drive mode
 */
if (HAL_PWREx_EnableOverDrive() != HAL_OK)
{
    Error_Handler();
}

```

```

/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                               |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

```

```

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
{
    Error_Handler();
}
}

/* USER CODE BEGIN 4 */

void process_command_message(uint8_t command) {
    if (command == '1' || command == '2') {
        sprintf(output_buffer, "Mode: %s\r\n", mode_name[current_mode]);
    } else if (command == '3' || command == '4') {
        sprintf(output_buffer, "Speed: %ld%%\r\n", speed_percents);
    } else if (command == '5') {
        sprintf(output_buffer, "Mode: %s\r\nSpeed: %ld%%\r\n", mode_name[current_mode], speed_percents);
    } else if (command == '\n') {
        sprintf(output_buffer, "Custom menu is open\n");
    } else if (command == 0x1B) {
        sprintf(output_buffer, "Main menu is open\n");
    }
    HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));
}

void process_main_menu_command(uint8_t command) {
    if (command == '1') {
        if (current_mode == EVERY_SIGNAL) {
            current_mode = BREATH;
        } else {
            current_mode++;
        }
    }
    } else if (command == '2') {

```

```

    if (current_mode == BREATH) {
        current_mode = EVERY_SIGNAL;
    } else {
        current_mode--;
    }
} else if (command == '3') {
    if (speed_percents <= 90) {
        speed_percents += 10;
    }
} else if (command == '4') {
    if (speed_percents >= 10) {
        speed_percents -= 10;
    }
} else if (command == '\n') {
    current_menu_state = CUSTOM;
}

process_command_message(command);
}

void process_custom_menu_input(uint8_t symbol) {
    static char custom_menu_buffer[256];
    static uint8_t custom_menu_index = 0;
    static _Bool is_first_input = 1;

    if (is_first_input) {
        sprintf(output_buffer, "Enter sequence: <color> <brightness> <smooth> <duration>\r\n");
        HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));
        is_first_input = 0;
        return;
    }
}

```

```
// Обработка ESC - выход из режима настройки
```

```
if (symbol == 0x1B) { // ESC

    current_menu_state = MAIN;

    is_first_input = 1;

    custom_menu_index = 0;

    memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));

    process_command_message(symbol);

    return;

}
```

```
// Обработка Backspaces
```

```
if (symbol == 0x08 || symbol == 0x7F) { // Backspace или Delete

    if (custom_menu_index > 0) {

        custom_menu_index--;

        custom_menu_buffer[custom_menu_index] = 0;

        // Отправляем последовательность для удаления символа в терминале

        char bs[] = "\b \b";

        HAL_UART_Transmit_IT(&huart6, bs, strlen(bs));

    }

    return;

}
```

```
// Обработка Enter - парсинг введенной строки
```

```
if (symbol == '\r' || symbol == '\n') {

    if (custom_menu_index == 0) {

        sprintf(output_buffer, "\r\nNo input! Press ESC to cancel or enter sequence: ");

        HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));

        return;

    }

}
```

```
// Парсим введенную строку
```

```

char color_char;

uint32_t brightness;

char smooth_char;

uint32_t duration;


int parsed = sscanf(custom_menu_buffer, "%c %lu %c %lu",
                    &color_char, &brightness, &smooth_char, &duration);


if (parsed != 4) {
    sprintf(output_buffer, "\r\nInvalid format! Use: <color> <brightness> <smooth> <duration>\r\n");
    HAL_UART_Transmit_IT(&huart6, (uint8_t*)output_buffer, strlen(output_buffer));
    custom_menu_index = 0;
    memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));
    return;
}


// Валидация цвета
indicator_color color;

if (color_char == 'r' || color_char == 'R') {
    color = RED;
} else if (color_char == 'y' || color_char == 'Y') {
    color = YELLOW;
} else if (color_char == 'g' || color_char == 'G') {
    color = GREEN;
} else {
    sprintf(output_buffer, "\r\nInvalid color '%c'! Use r, y or g\r\n", color_char);
    HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));
    custom_menu_index = 0;
    memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));
    return;
}

```

```
// Валидация яркости
```

```
if (brightness < 0 || brightness > 100) {  
    sprintf(output_buffer, "\r\nInvalid brightness %lu! Use 0-100\r\n", brightness);  
    HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));  
    custom_menu_index = 0;  
    memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));  
    return;  
}
```

```
_Bool is_smooth;
```

```
if (smooth_char == '+') {  
    is_smooth = 1;  
} else if (smooth_char == '-') {  
    is_smooth = 0;  
} else {  
    sprintf(output_buffer, "\r\nInvalid smooth '%c'! Use + or -\r\n", smooth_char);  
    HAL_UART_Transmit_IT(&huart6, (uint8_t*)output_buffer, strlen(output_buffer));  
    custom_menu_index = 0;  
    memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));  
    return;  
}
```

```
if (duration <= 0) {  
    sprintf(output_buffer, "\r\nInvalid duration %lu! Must be > 0\r\n", duration);  
    HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));  
    custom_menu_index = 0;  
    memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));  
    return;  
}
```

```
if (is_first_input) {  
    current_custom_mode_step_count = 0;
```

```

    is_first_input = 0;
}

if (current_custom_mode_step_count < MAX_CUSTOM_MODE_STEP_COUNT) {
    custom_mode_steps[current_custom_mode_step_count].color = color;
    custom_mode_steps[current_custom_mode_step_count].brightness = brightness;
    custom_mode_steps[current_custom_mode_step_count].is_smooth = is_smooth;
    custom_mode_steps[current_custom_mode_step_count].duration = duration;

    sprintf(output_buffer, "\r\nStep %d added: %s, %lu%%, %s, %lums\r\n",
        current_custom_mode_step_count + 1,
        (color == RED) ? "RED" : (color == YELLOW) ? "YELLOW" : "GREEN",
        brightness,
        is_smooth ? "smooth" : "instant",
        duration);
    HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));

    current_custom_mode_step_count++;

    if (current_custom_mode_step_count == MAX_CUSTOM_MODE_STEP_COUNT) {
        sprintf(output_buffer, "Maximum steps reached (%d). Press ESC to finish.\r\n",
MAX_CUSTOM_MODE_STEP_COUNT);
        HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));
        current_menu_state = MAIN;
        is_first_input = 1;

    } else {
        sprintf(output_buffer, "Enter next step or press ESC to finish: ");
        HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));
    }
} else {
    sprintf(output_buffer, "Maximum steps reached (%d). Press ESC to finish.\r\n",
MAX_CUSTOM_MODE_STEP_COUNT);

```



```

    HAL_UART_Transmit_IT(&huart6, output_buffer, strlen(output_buffer));

    current_menu_state = MAIN;

    is_first_input = 1;
}

custom_menu_index = 0;

memset(custom_menu_buffer, 0, sizeof(custom_menu_buffer));

return;
}

if (custom_menu_index < sizeof(custom_menu_buffer) - 1 && symbol >= 32 && symbol <= 126) {
    custom_menu_buffer[custom_menu_index++] = symbol;
    HAL_UART_Transmit_IT(&huart6, &symbol, 1);
}
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart == &huart6) {
        if (current_menu_state == MAIN) {
            process_main_menu_command(input_buffer);
        } else {
            process_custom_menu_input(input_buffer);
        }
        HAL_UART_Receive_IT(&huart6, &input_buffer, 1);
    }
}

void wait(uint32_t delay) {
    if (delay == 0) return;

    uint32_t start_tick = HAL_GetTick();
    while (HAL_GetTick() - start_tick < delay) {}
}

```

```

void play_breath() {
    static uint32_t brightness = 0;
    static uint32_t max_delay = 110;
    static _Bool going_up = 1;

    if (speed_percents == 0) {
        return;
    }

    htim4.Instance->CCR2 = brightness;
    htim4.Instance->CCR3 = 0;
    htim4.Instance->CCR4 = 0;

    // ❖СПРАВЛЕНО: инвертированная формула скорости
    // speed_percents=10 -> минимальная скорость, speed_percents=100 -> максимальная скорость
    uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
    wait(actual_delay);

    if (going_up) {
        brightness += 10;
        if (brightness >= MAX_BRIGHTNESS) {
            brightness = MAX_BRIGHTNESS;
            going_up = 0;
        }
    } else {
        brightness -= 10;
        if (brightness <= MIN_BRIGHTNESS) {
            brightness = MIN_BRIGHTNESS;
            going_up = 1;
        }
    }
}

```

```
}
```

```
void play_strobe() {
```

```
    static uint32_t max_delay = 1000;
```

```
    if (speed_percents == 0) {
```

```
        return;
```

```
    }
```

```
    htim4.Instance->CCR3 = 0;
```

```
    htim4.Instance->CCR4 = 0;
```

```
    htim4.Instance->CCR2 = 100;
```

```
//  СПРАВЛЕНО: инвертированная формула скорости
```

```
uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
```

```
wait(actual_delay);
```

```
    htim4.Instance->CCR2 = 0;
```

```
    wait(actual_delay);
```

```
}
```

```
void play_siren() {
```

```
    static uint32_t brightness = 0;
```

```
    static uint32_t max_delay = 110;
```

```
    static _Bool going_up = 1;
```

```
    if (speed_percents == 0) {
```

```
        return;
```

```
    }
```

```
    htim4.Instance->CCR2 = brightness;
```

```
    htim4.Instance->CCR3 = 100 - brightness;
```

```
htim4.Instance->CCR4 = 0;
```

```
// ⚡?СПРАВЛЕНО: инвертированная формула скорости
```

```
uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
```

```
wait(actual_delay);
```

```
if (going_up) {
```

```
    brightness += 10;
```

```
    if (brightness >= MAX_BRIGHTNESS) {
```

```
        brightness = MAX_BRIGHTNESS;
```

```
        going_up = 0;
```

```
    }
```

```
} else {
```

```
    brightness -= 10;
```

```
    if (brightness <= MIN_BRIGHTNESS) {
```

```
        brightness = MIN_BRIGHTNESS;
```

```
        going_up = 1;
```

```
    }
```

```
}
```

```
}
```

```
void play_every_signal() {
```

```
    static uint16_t pin = GPIO_PIN_13;
```

```
    static uint32_t max_delay = 1000;
```

```
    if (speed_percents == 0) {
```

```
        return;
```

```
    }
```

```
    if (pin == GPIO_PIN_13) {
```

```
        htim4.Instance->CCR2 = 100;
```

```
        htim4.Instance->CCR3 = 0;
```

```

    htim4.Instance->CCR4 = 0;

    pin = GPIO_PIN_14;
} else if (pin == GPIO_PIN_14) {
    htim4.Instance->CCR2 = 0;

    htim4.Instance->CCR3 = 100;

    htim4.Instance->CCR4 = 0;

    pin = GPIO_PIN_15;
} else {
    htim4.Instance->CCR2 = 0;

    htim4.Instance->CCR3 = 0;

    htim4.Instance->CCR4 = 100;

    pin = GPIO_PIN_13;
}

// ❖СПРАВЛЕНО: инвертированная формула скорости
uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
wait(actual_delay);
}

void play_custom_mode() {
    if (current_menu_state == CUSTOM) {
        return;
    }

    if (current_custom_mode_step_count == 0) {
        return;
    }

    custom_mode_step current_custom_mode_step = custom_mode_steps[current_custom_mode_step_index];

    if (current_custom_mode_step.color == RED) {
        htim4.Instance->CCR2 = 0;

```

```

        htim4.Instance->CCR3 = 0;

    htim4.Instance->CCR4 = current_custom_mode_step.brightness;

    } else if (current_custom_mode_step.color = YELLOW) {

        htim4.Instance->CCR2 = 0;

        htim4.Instance->CCR3 = current_custom_mode_step.brightness;

    htim4.Instance->CCR4 = 0;

    } else {

        htim4.Instance->CCR2 = current_custom_mode_step.brightness;

        htim4.Instance->CCR3 = 0;

    htim4.Instance->CCR4 = 0;

    }

    wait(current_custom_mode_step.duration);

    current_custom_mode_step_index++;

}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */

    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {

    }

    /* USER CODE END Error_Handler_Debug */
}

```

```

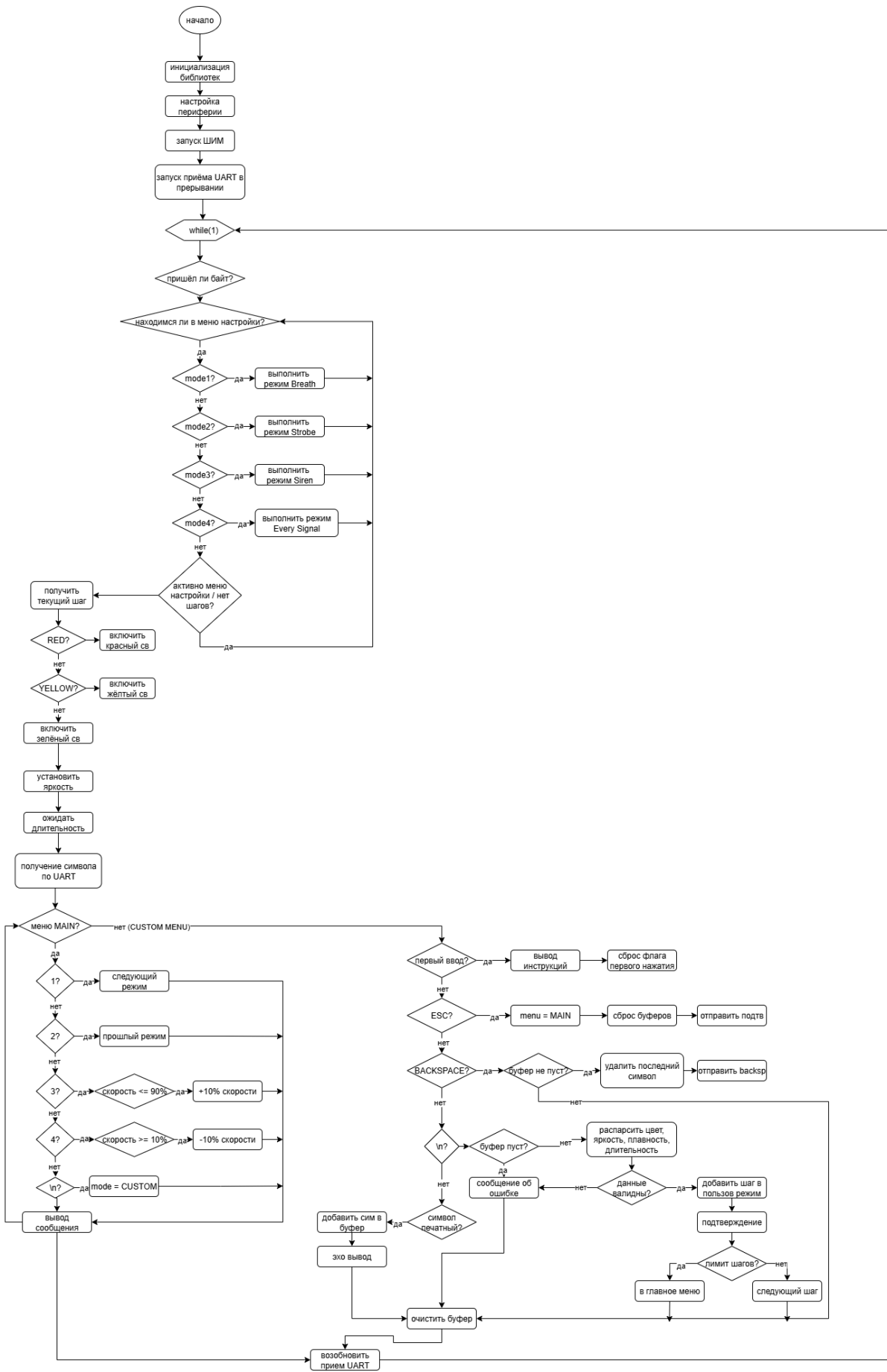
#ifdef USE_FULL_ASSERT

/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

#endif /* USE_FULL_ASSERT */

```

Блок-схема



Заключение

В ходе выполнения данной лабораторной работы удалось реализовать имитатор гирлянды с плавными включениями/выключениями, настройкой яркости и разными режимами, пользовательский режим.